我與老婆（Claude Code）的一天

Human-in-the-Loop AI 協作模式實戰案例

Jimmy Chen

為什麼叫「老婆」？

跟 Claude Code 協作，真的很像跟老婆相處：

- 要好好溝通，不能只丟一句話
- 她很聰明，但有時會搞錯你的意思
- 需要適時提醒和糾正
- 給對方向後，執行力超強
- 同樣的話問兩次，可能得到不同答案
- 聊太久她會忘記前面說過什麼

> *這不是在開玩笑，這是 AI 的真實特性。*

背景：那天發生什麼事

某個平凡的早上 11 點——

Gitea 被 OOM Killer 砍了。

連帶 supervisord 也被 systemctl 終止。
SSH 連不上，服務中斷。

我打開 Claude Code：

```
我：老婆，git-server 被 OOM killed 了，幫我查原因。
她：好，讓我 SSH 連進去... 連不上，SSH timeout 了。
    建議透過 AWS Console 重啟。
```

協作心法 #1：緊急狀況時先穩住

AI 很想幫忙，但機器連不上也沒辦法。

這時候人類要先處理基礎設施問題。

> *最厲害的 AI 給的建議，有時候跟 IT 部門的萬年 SOP 一樣：*
> *「Have you tried turning it off and on again?」*

AI 協作模式光譜

等級	名稱	運作方式
0	已知用火	人類全包，不用 AI
1	Copilot	AI 建議，人類採納
2	**HITL**	**人類決策 → AI 執行 → 人類審核**
3	Delegator	人類定義範圍，AI 自行完成
4	Supervisor	AI 執行，異常時通知人類
5	Autonomous	給目標，AI 完全自主

我們這次用的是 Level 2：Human-in-the-Loop

為什麼選 HITL？

- 這是**生產環境**，出錯代價很高
- 涉及 SSH、AWS、Terraform，需要人類判斷
- AI 會犯錯，需要即時糾正

HITL 核心原則

- 人類掌握決策權
- AI 負責執行
- 每個重要動作需要人類確認
- 可以即時糾正

> *沒有最好的模式，只有最適合的模式*

HITL × 護欄：人治 vs 法治

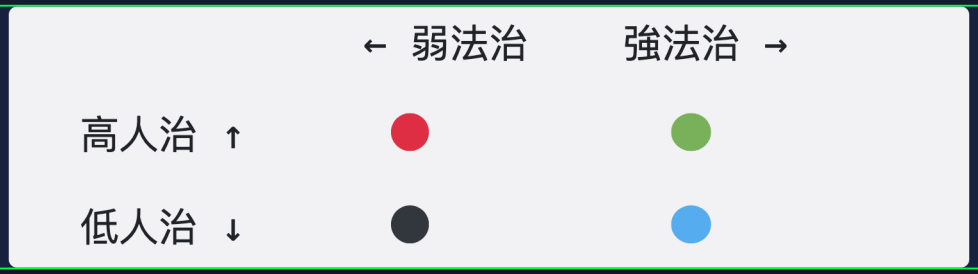
「我很好騙 對愛太渴望變成死穴 / 所有防備 全都防不了孤單侵略」

這首歌描述的是：**純靠意志力的脆弱狀態**。

治理類型	定義	弱點
人治	依賴人的意志力與判斷	累了就漏
法治	依賴規則與機制	需要事先設計好

關鍵：不是二選一，而是用法治來支撐人治。

HITL × 護欄：2×2 矩陣



象限	狀態	說明
🟢	穩健	高人治 + 強法治 = 雙重保護
🔴	脆弱	高人治 + 弱法治 = 純靠意志力
⚫	危險	低人治 + 弱法治 = 裸奔
🔵	自治	低人治 + 強法治 = 成熟目標

遷移路徑：🔴 → 🟢 → 🔵

實戰案例：七次糾正

案例一：不適合環境的建議

OOM 問題出現後，老婆第一個建議：

```
老婆：加 2GB Swap
我：   等等，AWS EC2 不是不建議開 swap 嗎？
老婆：對，EBS 的 I/O 延遲太高。那我們用其他方法。
```

協作心法 #2：質疑不適合環境的建議

AI 會給**通用建議**，但人類要判斷是否適合自己的環境。

> *這就像問老婆「晚餐吃什麼」，她說「吃火鍋」——*
> *但如果你正在減肥，你得自己判斷。*

案例二：子進程繼承問題

設定 `OOMScoreAdjust=-500` 保護 supervisord 後...

```
supervisord:    -500
gitea:          -500   ← 為什麼？
caddy:          -500
gitea-runner:   -500
```

所有子進程都繼承了父進程的 oom_score_adj！

原本想保護 supervisord，結果連 OOM 元凶 gitea 也被保護了。

協作心法 #4：實際部署後要驗證

> *理論上可行 ≠ 實際上可行*

案例三：換個思路

老婆：改成 -900 保護 supervisord，各子進程分別設定。
我：　等等，壓低 application 的 OOM score，
　　可能讓 kernel 去砍更底層的 sshd？
老婆：對，風險更高。
我：　那我們換個思路——拉高 gitea 讓它更容易被砍？
老婆：對！這是更合理的做法。

做法	問題
保護 supervisord（-900）	可能害死其他服務
拉高 gitea（+200）	無副作用

協作心法 #5：考慮副作用和合理性

案例四：先查既有模式

用 Terraform 加一個 40GB EBS...

Terraform plan 顯示：**instance must be replaced** 😱

```
# ❌ 錯誤：會替換 instance
ebs:
  - volume_type: gp3

# ✅ 正確：獨立的 volume attachment
ebs_volumes:
  - resource_name: backup
```

協作心法 #7：先查既有模式再動手

> *「我們之前怎麼做的？」這句話可以避免很多災難。*

案例五：工具能力確認

```
我：  用 tag-based lifecycle rule，這是我們的慣例。
老婆：好，讓我設定...
我：  等等，s5cmd 有支援設定 tag 嗎？
老婆：...不支援。
```

只能改用 prefix-based lifecycle。

協作心法 #10：確認工具能力

> *即使是人類指定的做法，也要確認工具是否支援。*

案例六 & 七

排程衝突

```
老婆：備份排程設 0 18 * * *
我：  gitea health check 也是 18:00，會撞車！
老婆：對欸！我改成 20:00。
```

協作心法 #13：跨系統衝突需要人類把關

安全把關

```
我：  等等，group_vars 裡面有 token，應該從一開始就移掉。
老婆：好，我用 filter-branch 重寫歷史。
```

協作心法 #14：AI 不會主動檢查是否 commit 了敏感資訊

讓 AI 操作 CLI 真的安全嗎？

控制的錯覺 (Illusion of Control)

心理學家 Ellen Langer 發現：自己選彩票號碼時，會對中獎更有信心——即使機率完全一樣。

控制的錯覺	真正的控制
我按了 Enter	我理解這個指令在做什麼
我選了這個選項	我能驗證結果是否正確
我親手打的字	我能在出錯時即時修正

安全性來自「指令正確 + 有審核機制」，不是「誰按 Enter」。

58 次中斷的數據

這一天我總共中斷了 AI **58 次**。

中斷原因	佔比
想先看計畫	30%
自己手動做	25%
補充環境脈絡	20%
驗證先行	15%
遵循團隊慣例	10%

0 次因為 AI 要執行「危險操作」
0 次因為 AI「完全搞錯方向」
0 次造成「實際損害」

> *風險的真正來源不是「AI 會亂來」，而是「人類是否保持關注」。*

把 AI 當老婆？小心誤區

老婆的特質	AI 的實際情況	風險
記得你	每次 session 都是新的	省略重要脈絡
會成長	不會從互動中學習	期待落空
有責任感	出事 AI 不負責	推卸責任
在乎你	只是在執行指令	情感依賴
會反對你	傾向順從	缺乏制衡

正確用法

相處方式學老婆：**耐心溝通、適時糾正**

但別忘了：她說的要驗證、出事你要扛、她不記得上次聊什麼

結論

這一天的成果

指標	數據
協作時間	7.5 小時
產出程式碼	200+ 行
協作心法	14 個
糾正次數	7 次

如果 AI 是人類，這大概是會讓她想離職的一天。
但她不是，所以她只是說：**「好的，我來修改。」**

核心結論

AI 是很強的執行者，

但需要人類當導演。

HITL 不是因為我們不信任 AI，而是因為：

1. AI 會犯錯，需要即時糾正
2. AI 不知道你的環境和慣例
3. AI 專注單一任務，缺乏系統視角
4. 出事的時候，是人類要負責

Q&A

謝謝大家

> *「明天見，老婆。」*

Demo: jimc1682000.github.io/a-day-with-my-ai-wife